Web Scraping Exercise

Web Scraping allows you to gather large volumes of data from diverse and real-time online sources. This data can be crucial for enriching your datasets, filling in gaps, and providing current information that enhances the quality and relevance of your analysis. Web scraping enables you to collect data that might not be readily available through traditional APIs or databases, offering a competitive edge by incorporating unique and comprehensive insights. Moreover, it automates the data collection process, saving time and resources while ensuring a scalable approach to continuously updating and maintaining your datasets.

Ethical web scraping involves respecting website terms of service, avoiding overloading servers, and ensuring that the collected data is used responsibly and in compliance with privacy laws and regulations.

Use Python, `requests`, `BeautifulSoup` and/or `pandas` to scrape web data:

Import Libraries

In [1]:
```
# Import libraries for HTTP requests, HTML parsing, data handling, and file paths.
from pathlib import Path
import re

import pandas as pd
import requests
from bs4 import BeautifulSoup

# Show a short confirmation that the required libraries are ready.
print("Libraries imported: requests, BeautifulSoup, pandas, pathlib, re.")
```

```
Libraries imported: requests, BeautifulSoup, pandas, pathlib, re.
```

Define the Target URL

In [2]:
```
# Store the Wikipedia page URL that will be scraped.
url = "https://en.wikipedia.org/wiki/List_of_European_Union_cities_proper_by_popula

# Print the selected source so the scraping target is documented in the output.
print(f"Target URL set to: {url}")
```

```
Target URL set to: https://en.wikipedia.org/wiki/List_of_European_Union_cities_prope
r_by_population_density
```

Send a Request to the Website

Do not forget to check the response status code

In [3]:
```
# Define a polite user agent for this educational scraping request.
headers = {"User-Agent": "Mozilla/5.0 (educational web scraping exercise)"}

# Send a GET request to download the web page HTML.
response = requests.get(url, headers=headers, timeout=20)

# Stop the notebook if the website did not return a successful response.
response.raise_for_status()

# Print the HTTP status and response size to confirm that the request worked.
print(f"Request successful: status code {response.status_code}.")
print(f"Downloaded HTML size: {len(response.text):,} characters.")
```

```
Request successful: status code 200.
Downloaded HTML size: 116,587 characters.
```

Parse the HTML Content

Use a library to access the HTMl content

In [4]:
```
# Parse the downloaded HTML with BeautifulSoup.
soup = BeautifulSoup(response.text, "html.parser")

# Find all Wikipedia tables that use the standard wikitable class.
tables = soup.find_all("table", class_="wikitable")

# Use the first wikitable because this page contains the city density ranking there
target_table = tables[0]

# Print a short parsing summary.
print(f"HTML parsed successfully and {len(tables)} wikitable table(s) found.")
print("The first wikitable was selected for extraction.")
```

```
HTML parsed successfully and 1 wikitable table(s) found.
The first wikitable was selected for extraction.
```

Identify the Data to be Scraped

Write a couple of sentence on the data you want to scrape

I want to scrape the ranking table of European Union cities proper by population density from Wikipedia. The dataset contains each city's rank, name, population, area, density, and country. After extraction, I will clean numeric columns so the data can be analyzed and saved as a structured CSV file.

Extract Data

Find specific elements and extract text or attributes from elements (handle pagination if necessary)

In [5]:
```python
# Read the table headers from the first row of the selected table.
headers = [cell.get_text(" ", strip=True) for cell in target_table.find_all("th")]

# Create a list for the extracted row values.
rows = []

# Loop through all data rows after the header row.
for table_row in target_table.find_all("tr")[1:]:
    # Extract text from each table cell in the current row.
    values = [cell.get_text(" ", strip=True) for cell in table_row.find_all("td")]

    # Keep only complete rows that match the number of headers.
    if len(values) == len(headers):
        rows.append(values)

# Build a DataFrame from the extracted rows and headers.
raw_df = pd.DataFrame(rows, columns=headers)

# Rename columns to simple snake_case names for easier analysis.
clean_df = raw_df.rename(columns={
    "Rank": "rank",
    "City": "city",
    "Population": "population",
    "Area (km 2 )": "area_km2",
    "Area (sq. miles)": "area_sq_miles",
    "Density (/km 2 )": "density_per_km2",
    "Density (/sq. mile)": "density_per_sq_mile",
    "Country": "country",
})

# Define the columns that should be converted from text to numeric values.
numeric_columns = ["rank", "population", "area_km2", "area_sq_miles", "density_per_

# Remove citation markers and thousands separators before numeric conversion.
for column in numeric_columns:
    clean_df[column] = clean_df[column].str.replace(r"\[.*?\]", "", regex=True)
    clean_df[column] = clean_df[column].str.replace(",", "", regex=False)
    clean_df[column] = pd.to_numeric(clean_df[column], errors="coerce")

# Strip extra whitespace from text columns.
clean_df["city"] = clean_df["city"].str.strip()
clean_df["country"] = clean_df["country"].str.strip()

# Sort by rank so duplicate city-country entries keep the highest-ranked source row
clean_df = clean_df.sort_values("rank").drop_duplicates(subset=["city", "country"],

# Reset the row index after duplicate cleanup.
clean_df = clean_df.reset_index(drop=True)

# Print a concise result and show the first rows of the cleaned data.
print(f"Extracted and cleaned {len(clean_df)} city records after removing duplicate
print("Preview of the cleaned dataset:")
display(clean_df.head())
```

```
Extracted and cleaned 68 city records after removing duplicate city-country rows.
Preview of the cleaned dataset:
```

	rank	city	population	area_km2	area_sq_miles	density_per_km2	density_per_sq_r
0	1	Levallois-Perret	66082	2.41	0.93	27420	71
1	2	Emperador	692	0.03	0.01	23067	69
2	3	L'Hospitalet de Llobregat	264923	12.40	4.80	21364	55
3	4	Paris	2203817	105.40	40.50	20909	54
4	5	Mislata	43278	2.10	0.81	20608	53

Store Data in a Structured Format

Give a brief overview of the data collected (e.g. count, fields, ...)

In [6]:
```
# Count missing values per column to check data quality.
missing_values = clean_df.isna().sum()

# Count duplicate city-country combinations to check uniqueness.
duplicate_city_country_rows = clean_df.duplicated(subset=["city", "country"]).sum()

# Print a compact overview of the structured dataset.
print(f"Rows: {clean_df.shape[0]}")
print(f"Columns: {clean_df.shape[1]}")
print(f"Duplicate city-country rows: {duplicate_city_country_rows}")
print("Missing values per column:")
print(missing_values.to_string())

# Display the ten densest cities as a small validation sample.
print("Top 10 cities by population density per km2:")
display(clean_df.sort_values("density_per_km2", ascending=False).head(10))
```

```
Rows: 68
Columns: 8
Duplicate city-country rows: 0
Missing values per column:
rank                  0
city                  0
population            0
area_km2              0
area_sq_miles         0
density_per_km2       0
density_per_sq_mile   0
country               0
Top 10 cities by population density per km2:
```

	rank	city	population	area_km2	area_sq_miles	density_per_km2	density_per_sq_r
0	1	Levallois-Perret	66082	2.41	0.93	27420	71
1	2	Emperador	692	0.03	0.01	23067	69
2	3	L'Hospitalet de Llobregat	264923	12.40	4.80	21364	55
3	4	Paris	2203817	105.40	40.50	20909	54
4	5	Mislata	43278	2.10	0.81	20608	53
5	6	Benetússer	14668	0.76	0.29	19300	50
6	7	Athens	745514	38.96	15.04	19135	49
7	8	Sliema	22591	1.30	0.50	17377	45
8	9	Thessaloniki	325182	19.31	7.45	16840	43
9	10	Barcelona	1621537	101.90	39.30	15991	41

Save the Data

In [7]:
```
# Define an output directory next to this notebook.
output_dir = Path("data")

# Create the output directory if it does not exist yet.
output_dir.mkdir(exist_ok=True)

# Define the CSV output file path.
output_file = output_dir / "eu_city_population_density.csv"

# Save the cleaned DataFrame as a UTF-8 CSV file without the DataFrame index.
clean_df.to_csv(output_file, index=False, encoding="utf-8")

# Print the saved file path and size to confirm the export.
print(f"Cleaned data saved to: {output_file}")
print(f"Saved file size: {output_file.stat().st_size:,} bytes.")
```

```
Cleaned data saved to: data\eu_city_population_density.csv
Saved file size: 3,539 bytes.
```